SECURE CODING REVIEW REPORT

Title:
Authentication Security Review in Python Application

Prepared By:
Maham Riaz

Date:
31/05/2026

1. INTRODUCTION

Secure coding practices are essential for protecting applications against security threats. This review analyzes a Python-based login application and identifies vulnerabilities related to authentication and credential management.

2. OBJECTIVE

The objective of this review is to:

- Identify security vulnerabilities in the application.
- Assess the associated risks.
- Recommend secure coding improvements.
- Demonstrate a secure implementation.

3. APPLICATION OVERVIEW

The application performs user authentication by comparing a username and password against predefined credentials.

The reviewed application contains several security weaknesses that could expose user accounts to unauthorized access.

4. CODE REVIEW FINDINGS

Vulnerability 1: Hardcoded Credentials

Description:
The username and password are stored directly inside the source code.

Example:

USERNAME = "admin"
PASSWORD = "admin123"

Risk Level:
High

Impact:
• Credentials can be exposed if the source code is leaked.
• Attackers can easily discover login information.
• Difficult to manage credential updates securely.

Vulnerability 2: Weak Password Policy

Description:
The password "admin123" is predictable and commonly used.

Risk Level:
Medium

Impact:
• Increased susceptibility to brute-force attacks.
• Reduced overall account security.

Vulnerability 3: Plaintext Password Handling

Description:
Passwords are stored and compared in plaintext form.

Risk Level:
High

Impact:
• Passwords can be exposed if the application is compromised.
• Violates standard authentication security practices.

5. RISK ASSESSMENT

The identified vulnerabilities could allow unauthorized users to gain access to the application. Exposure of credentials may lead to account compromise, data disclosure, and loss of confidentiality.

Overall Risk Rating:
HIGH

6. RECOMMENDED REMEDIATION

The following improvements are recommended:

- Remove hardcoded passwords.
- Store password hashes instead of plaintext passwords.
- Use strong password policies.
- Implement secure credential storage mechanisms.
- Regularly review authentication controls.

7. SECURE IMPLEMENTATION

A secure version of the application was developed using SHA-256 password hashing.

Security Improvements:

- Passwords are not stored in plaintext.
- Authentication uses hashed password comparison.
- Improved protection against credential disclosure.

8. TOOLS USED

- Python 3
- hashlib Library
- Manual Secure Code Review

9. CONCLUSION

The review identified multiple authentication-related security weaknesses, including hardcoded credentials, weak passwords, and plaintext password handling.

By implementing password hashing and following secure coding practices, the application's security posture is significantly improved. Secure authentication mechanisms help reduce the risk of unauthorized access and strengthen overall system security.